Tarea: Perceptrón Multicapa

Ciencia de Datos — Universidad Católica del Norte

Objetivos de esta tarea

Al terminar esta tarea vas a poder:

- Construir un **perceptrón multicapa** con `torch.nn`, eligiendo profundidad, ancho y activación, y entender por qué la capa de salida va *sin* activación cuando la pérdida es entropía cruzada.
- Escribir el **bucle de entrenamiento** completo (pasada hacia adelante, pérdida, `zero_grad`, `backward`, `step`) y su contraparte de **evaluación**, que no aprende sino que mide.
- Distinguir en el código lo que cambia entre *entrenar* y *medir*: modo del modelo, gradientes y actualización de pesos.
- Diseñar y ejecutar un **estudio de ablación** de un factor a la vez sobre tasa de aprendizaje, tamaño de lote, activación y profundidad, y leer sus resultados en una tabla de exactitud de entrenamiento y de validación.

Contexto

En la sesión sobre aprendizaje de redes neuronales planteamos el aprendizaje de una red como un problema de minimización:

$$\text{TrainLoss}(W) \;=\; \frac{1}{N}\sum_{i=1}^{N} \operatorname{div}\left(f(X_i; W),\, d_i\right), \qquad \widehat{W} \;=\; \operatorname*{argmin}_{W} \text{TrainLoss}(W).$$

Para clasificación multiclase vimos que el objetivo d_i se representa *one-hot*, que la capa de salida es una *softmax* y que la divergencia es la **entropía cruzada**, que con objetivos *one-hot* se reduce a $\operatorname{div}(Y, d) = -\log y_c$, donde c es la clase correcta. Ese mínimo se busca con descenso por gradiente (hoy, sobre lotes de ejemplos) y los gradientes los entrega *backpropagation*.

Hoy nada de eso hay que programarlo a mano: PyTorch trae las capas, la entropía cruzada y el optimizador. Lo que sí hay que programar es **todo lo que los rodea**, que es donde de verdad se equivoca uno. Y después viene la parte interesante: la red entrena, pero ¿qué pasa si la tasa de aprendizaje es cien veces más pequeña? ¿Y si la activación es una sigmoide en vez de una *ReLU*? Eso se responde con un experimento, no con una opinión, y ese experimento se llama **estudio de ablación**.

Los datos y el tiempo de cómputo. Clasificamos dígitos manuscritos de 28×28 píxeles en diez clases. `util.py` descarga MNIST con `torchvision` a la carpeta `datos/` de la tarea la primera vez que lo ejecutas (unos 11 MB); si el computador no tiene red o no tiene `torchvision`, genera localmente dígitos sintéticos de siete segmentos con inclinación, desplazamiento y ruido, y te avisa por consola cuál de las dos fuentes está usando. Las conclusiones de la tarea son las mismas en los dos casos.

Para que la tarea corra en cualquier computador, se entrena sobre un **subconjunto** de 6.000 imágenes de entrenamiento y 2.000 de validación, y con pocas épocas. Una época demora un par de segundos en el procesador, y `experimento.py` completo (las dos arquitecturas más las diez configuraciones del estudio de ablación) demora menos de un minuto con los datos ya descargados; a la primera

ejecución hay que sumarle la descarga de MNIST (unos 11 MB). Nada de esto usa tarjeta gráfica ni servicios en línea.

Cómo trabajar

Necesitas Python 3, numpy y PyTorch (`pip install -r requirements.txt`). El único archivo que se edita es `submission.py`: busca los bloques marcados con `# BEGIN_TU_CODIGO`. Los demás archivos no se tocan.

Archivo	Qué es	¿Se edita?
`submission.py`	la red, el bucle y la ablación	sí
`util.py`	datos, cargadores, semilla y CSV	no
`experimento.py`	el cierre: entrena y arma la tabla	no
`grader.py`	autograder, corre sin red	no

Cada vez que completes una parte, corre el autograder desde la carpeta de la tarea:

```
python grader.py
```

Imprime el detalle prueba por prueba y el subtotal de cada parte. El corrector que viene con la tarea comprueba una parte del puntaje: llegar a su total completo es condición necesaria y no suficiente, porque la nota la pone el corrector del curso, que ejecuta esas mismas pruebas más otras sobre lo mismo. Por eso el subtotal que imprime el autograder en cada problema puede quedar por debajo del puntaje que encabeza el problema en este documento; el propio autograder lo aclara al pie de cada parte. Los 100 puntos son automáticos: no hay nada que se corrija a mano.

El cierre de la tarea es `experimento.py`, que entrena, corre el estudio de ablación e imprime la tabla de resultados lista para copiar en `submission.py`. El orden para terminar es: resuelve las cuatro partes, corre `experimento.py`, pega la tabla y vuelve a correr el autograder.

Problema 1: La arquitectura **[20 puntos]**

Un perceptrón multicapa es una pila de capas afines separadas por activaciones. En `torch.nn` eso es literalmente una lista de módulos dentro de un `nn.Sequential`.

(a) Completa `tamanos_ocultos(forma, ancho, profundidad)`, que devuelve la lista con el número de neuronas de cada capa oculta. Las dos formas de la tarea son:

- **cilíndrica**: todas las capas ocultas tienen el mismo ancho. `("cilindrica", 128, 3)` da `[128, 128, 128]`.
- **piramidal**: cada capa tiene la mitad de neuronas que la anterior, con un piso de una neurona. `("piramidal", 128, 3)` da `[128, 64, 32]`.

Con una forma que no esté en `FORMAS`, levanta `ValueError`.

(b) Completa `crear_mlp(tamano_entrada, ocultas, tamano_salida, activacion)`. El orden es: `nn.Flatten()` para estirar la imagen a un vector de 784 componentes; por cada tamaño de `ocultas` una `nn.Linear` seguida de la activación; y una `nn.Linear` final hacia las 10 clases **sin activación**. La activación se elige por nombre desde el diccionario `ACTIVACIONES`; si el nombre no está, levanta `ValueError`.

(c) Completa `contar_parametros(modelo)`: la suma de elementos de todos los parámetros entrenables.

Pista

La capa de salida va sin activación **a propósito**. `F.cross_entropy` espera los puntajes crudos (los *logits*) porque aplica internamente el `log`-*softmax*: es más estable numéricamente que calcular la *softmax* y después su logaritmo. Si además le pones una `nn.Softmax` al final, estarías aplicándola dos veces y la red entrena mucho peor.
Para instanciar la activación, recuerda que `ACTIVACIONES[nombre]` es la *clase* (por ejemplo `nn.ReLU`), así que hay que llamarla: `ACTIVACIONES[nombre]()`. Y como cada capa oculta necesita una instancia propia, la llamada va dentro del ciclo.

Problema 2: El bucle de entrenamiento y la evaluación [26 puntos]

Aquí está el corazón de la tarea. Son dos funciones cortas, casi gemelas, y todo el aprendizaje está en las diferencias entre ellas.

(a) Completa `entrenar_una_epoca(modelo, cargador, optimizador)`. Deja el modelo en modo entrenamiento con `modelo.train()` y, para cada lote `(x, y)`:

1. `puntajes = modelo(x);`
2. `perdida = F.cross_entropy(puntajes, y);`
3. `optimizador.zero_grad();`
4. `perdida.backward();`
5. `optimizador.step()`.

Devuelve `(perdida_media, exactitud)` sobre toda la época.

(b) Completa `evaluar(modelo, cargador)`: lo mismo, pero con `modelo.eval()`, todo el recorrido dentro de `with torch.no_grad():` y **sin** `backward` ni `step`. Medir no es aprender.

Sobre el promedio. `F.cross_entropy` devuelve el promedio *dentro del lote*. Para obtener la pérdida media del conjunto completo hay que ponderar cada lote por su tamaño:

$$\text{pérdida media} \;=\; \frac{\sum_b |b| \cdot \text{pérdida}(b)}{\sum_b |b|},$$

porque el último lote casi nunca está completo. Promediar sin ponderar le da al último lote el mismo peso que a los demás y entrega un número distinto. La exactitud se cuenta igual: aciertos acumulados dividido por ejemplos vistos, con `puntajes.argmax(dim=1) == y`.

Pista

`perdida.item()` convierte el tensor de un elemento a un `float` de Python y, de paso, corta la referencia al grafo de computación: si acumulas el tensor en vez del `float`, el grafo entero queda vivo en memoria toda la época.
`x.size(0)` es el número de ejemplos del lote.

Problema 3: Entrenar de punta a punta [24 puntos]

Completa `entrenar_modelo(modelo, cargador_entrenamiento, cargador_validacion, epocas, tasa)`.

Crea **un solo** optimizador antes del ciclo, `torch.optim.SGD(modelo.parameters(), lr=tasa)`, y en cada época: entrena con `entrenar_una_epoca`, mide con `evaluar` sobre el cargador de validación y

agrega al historial un diccionario con las claves `epoca` (desde 1), `perdida_entrenamiento`, `exactitud_entrenamiento`, `perdida_validacion` y `exactitud_validacion`. Devuelve la lista de largo `epocas`.

Usamos descenso por gradiente estocástico puro, sin momento y sin `Adam`, para que el efecto de la tasa de aprendizaje se vea sin nada que lo amortigüe: es justamente lo que vamos a variar en el Problema 4.

Pista

El optimizador guarda referencias a los parámetros del modelo, así que hay que crearlo *después* del modelo y una sola vez. Crearlo dentro del ciclo de épocas no rompe nada con SGD puro, pero con cualquier optimizador con estado (como `Adam`) se perdería ese estado en cada época.

Problema 4: El estudio de ablación [30 puntos]

Un estudio de ablación responde una pregunta a la vez: se fija una configuración de referencia y se cambia **un solo** hiperparámetro cada vez. Si se cambian dos, cualquier diferencia observada ya no se le puede atribuir a ninguno.

En `submission.py` ya están la configuración base y la lista de variaciones:

fila	qué cambia	valor
`base`	—	`cilindrica`, ancho 128, prof. 2, `relu`, $\eta = 0{,}1$, lote 64
`tasa-baja`	`tasa`	$0{,}001$
`tasa-alta`	`tasa`	$1{,}5$
`lote-pequeño`	`lote`	16
`lote-grande`	`lote`	256
`activacion-sigmoide`	`activacion`	`sigmoide`
`activacion-tanh`	`activacion`	`tanh`
`profundidad-1`	`profundidad`	1
`profundidad-4`	`profundidad`	4
`forma-piramidal`	`forma`	`piramidal`

(a) Completa `configuraciones_ablacion()`: devuelve una copia de `CONFIGURACION_BASE` seguida de una copia por cada entrada de `VARIACIONES_ABLACION`, con `nombre` y la clave correspondiente reemplazados. Cada configuración debe ser un diccionario **independiente**.

(b) Completa `ejecutar_ablacion(configuraciones, datos, ruta_csv, semilla)`. Para cada configuración, en este orden:

1. `util.fijar_semilla(semilla)`;
2. `util.crear_cargadores(datos, configuracion["lote"], semilla=semilla)`;
3. `tamanos_ocultos` y `crear_mlp` con lo que dice la configuración, usando `util.TAMANO_ENTRADA` y `util.TAMANO_SALIDA`;
4. `entrenar_modelo` con sus `epocas` y su `tasa`;
5. una fila con todas las columnas de `util.COLUMNAS_CSV`: los ocho valores de la configuración, `parametros` y las cuatro métricas de la **última** época.

Al final, `util.escribir_csv(filas, ruta_csv)` y devuelve las filas.

Los pasos 1 y 2 no son un detalle, y hay que hacer los dos. El paso 1 fija la **inicialización de los pesos**: si cada corrida partiera de pesos distintos, parte de la diferencia entre dos filas vendría del azar y la tabla dejaría de significar algo. El paso 2 fija el **orden del barajado**: `crear_cargadores` sortea los lotes con un generador propio sembrado con esa semilla, en vez de tomarlos del generador global de torch. Si los tomara del global, su punto de partida dependería de cuántos números al azar consumió

antes la construcción del modelo (que es distinta para cada arquitectura) y entonces `profundidad-1` y `forma-piramidal` verían los ejemplos en otro orden que la base. Esa sería una segunda diferencia, y una ablación con dos variables sueltas a la vez no prueba nada. Con los dos pasos, la única diferencia entre dos filas es el hiperparámetro que cambió.

Cuando termines, ejecuta el cierre:

```
python experimento.py
```

Entrena las dos arquitecturas, corre las diez configuraciones, deja la tabla en `ablacion.csv` (un archivo de trabajo, que **no** se entrega) y la imprime además ya escrita como constante de Python.

(c) Copia esa constante en la sección **Tus respuestas** de `submission.py`, reemplazando `TABLA_ABLACION`: una tupla por fila, en el orden del estudio, con los trece valores en el orden de `util.COLUMNAS_CSV`. Vale **5 de los 30 puntos** de este problema.

La revisa el corrector del curso: que estén las diez filas en el orden del estudio y con su nombre, que cada una declare los hiperparámetros de su configuración y la columna `parametros` exacta, que exactitudes y pérdidas estén en rango, y que la tabla sea coherente consigo misma (que la fila base aprenda, que las dos tasas extremas queden muy por debajo, que la sigmoide rinda peor y el lote pequeño mejor que el grande). Y, sobre todo, compara las cuatro métricas entrenadas de cada fila con la corrida de referencia del curso, que usa estas mismas configuraciones, la misma semilla y los mismos datos que arma `util.py`. La comparación deja holgura para que otro computador u otra versión de PyTorch muevan los decimales, y sirven por igual las dos fuentes de datos (MNIST y los dígitos sintéticos, según lo que tu computador haya podido obtener). Una tabla escrita a mano, por coherente que se vea, no pasa: los números salen de correr el experimento. Si dejas `TABLA_ABLACION` vacía, esos 5 puntos se pierden.

Pista

`dict(CONFIGURACION_BASE)` crea una copia superficial, que aquí basta porque todos los valores son números y textos. `configuracion = base` no copia nada: deja dos nombres apuntando al mismo diccionario, y al modificar uno se modifican todos.

Preguntas para pensar. **No puntúan**: no se entregan, no se revisan y no descuentan. Están para que la tabla que acabas de producir signifique algo. Con tu tabla a la vista:

1. ¿Qué le pasó a la fila `tasa-alta`? ¿Por qué la exactitud queda cerca de $0{,}1$ y no en cero?
2. `lote-pequeño` y `lote-grande` corren las mismas dos épocas sobre los mismos datos, y aun así rinden distinto. ¿Cuántas actualizaciones de pesos alcanza a hacer cada uno en una época? La compensación que hay detrás la formalizaremos al estudiar el descenso por gradiente estocástico por lotes.
3. La red con `sigmoide` queda muy por detrás de la de `relu` y la de `tanh`. Mira la derivada de la sigmoide, $\sigma'(z) = \sigma(z)\,(1-\sigma(z))$: ¿cuánto vale como máximo, y qué le pasa al gradiente cuando esa derivada se multiplica capa a capa al retropropagar?
4. `profundidad-4` tiene más parámetros que la base y aun así rinde peor. ¿Contradice eso lo que vimos sobre aproximación universal?

Qué entregar

Un solo archivo, por Campus Virtual: `submission.py`, con los bloques `# BEGIN_TU_CODIGO` completados y con `TABLA_ABLACION` llena en la sección **Tus respuestas**. Nada más: ni `ablacion.csv`, ni la carpeta `datos/` (pesa varios megabytes y se regenera sola), ni los demás archivos de la tarea.
Los 100 puntos los dan los correctores automáticos; no hay nada que se corrija a mano.
Antes de entregar, corre `python experimento.py` y después `python grader.py`, en ese orden, y revisa que el informe llegue hasta el final sin fallas. `submission.py` debe importarse sin efectos secundarios: no dejes código suelto fuera de las funciones.